ANALISIS ACTIVIDAD_2

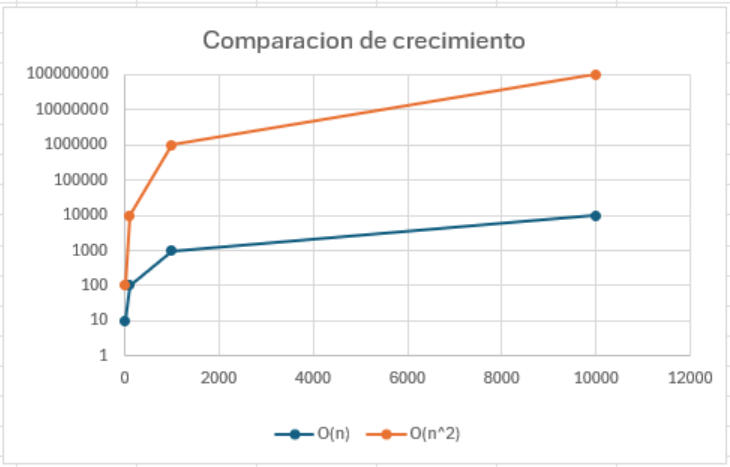
1. Clasificación de los fragmentos

Fragmento	Operaciones aproximadas	Big O	Justificación
Acceder a vector[5]	uno	O(1)	Solo accede a un dato independientemente de cuantos datos se tengan
Recorrer un arreglo	Uno por uno o dependiendo del numero de elementos que tenga el arreglo	O(n)	El ciclo recorre el numero de elementos que tenga el arreglo uno por uno hasta llegar al final
Dos ciclos anidados	exponencial o dependiendo del número de elementos que tenga el arreglo	O(n^2)	Como se revisan dos arreglos uno dentro de otro la carga se duplica
Dividir n entre 2 repetidamente	Logarítmica o dependiendo del número de elementos que tenga el arreglo	O(log n)	Cada vez el numero total de datos se va dividiendo a la mitad
Ciclo n + reducción logarítmica	Se ejecuta en completo el ciclo for y se divide en ciclo while	O(n log n)	Se ejecuta en completo el ciclo for y se divide en ciclo while

2. Tabla de tiempos

Tamaño n	Tiempo O(n)	Tiempo O(n²)	¿Cuál fue más rápido?
100	2600	68100	O(n)
1.000	16800	2322000	O(n)
5.000	64500	8954800	O(n)
10.000	83400	5700 (Anomalia)	O(n^2)

3. Grafica comparativa



n	O(n)	O(n^2)
10	10	100
100	100	10000
1000	1000	1000000
10000	10000	100000000

4. Respuestas preguntas de análisis

1. ¿Qué diferencia existe entre tiempo de ejecución y complejidad Big O?

Big O no mide el tiempo de ejecución sino lo complejo o cuantos procesos le toma a un algoritmo resolver la consulta

2. ¿Por qué O(1) se considera una complejidad eficiente?

Porque hace una única iteración, tiene focalizado donde se encuentra el dato

3. ¿Qué ocurre con un algoritmo O(n) cuando duplicamos el tamaño de entrada?

Duplica linealmente el número de iteraciones que hace

4. ¿Qué ocurre aproximadamente con un algoritmo O(n²) cuando duplicamos n?

Se cuadriplica la carga, hace que el número de iteraciones suba exponencialmente

5. ¿Por qué los ciclos anidados suelen relacionarse con O(n²)?

Porque está duplicando las iteraciones que se hacen

6. ¿Qué característica permite reconocer un algoritmo O(log n)?

Que tiene un bucle While

7. ¿Cuál de los dos algoritmos experimentales presentó mayor crecimiento?

El O(n²) → crecimiento rápido; crece exponencialmente

8. ¿Los resultados medidos fueron exactamente iguales entre las diferentes ejecuciones? ¿Por qué?

No, el único que presento igualdad en los resultados fue el O(1) que es constante

9. Si tuviera que procesar diez millones de registros, ¿qué tipo de complejidad preferiría?

O(log n) porque reduce a la mitad el espacio de búsqueda o el tamaño del problema en cada paso

10. ¿Por qué un ingeniero de software debe analizar la eficiencia antes de seleccionar un algoritmo?

Porque puede hacer el algoritmo mucho más eficiente con menos pasos

5. Reto adicional

```
for (int i = 0; i < n; i++) {
   System.out.println(i);
}

for (int j = 0; j < n; j++) {
   System.out.println(j);
}
```

Pregunta: ¿Es O(n^2)? Justifique su respuesta. Pista: los ciclos son consecutivos, no están anidados.

No es O(**n^2**) porque el segundo bucle no esta anidado dentro del primero por lo que sus iteraciones van a ser independientes